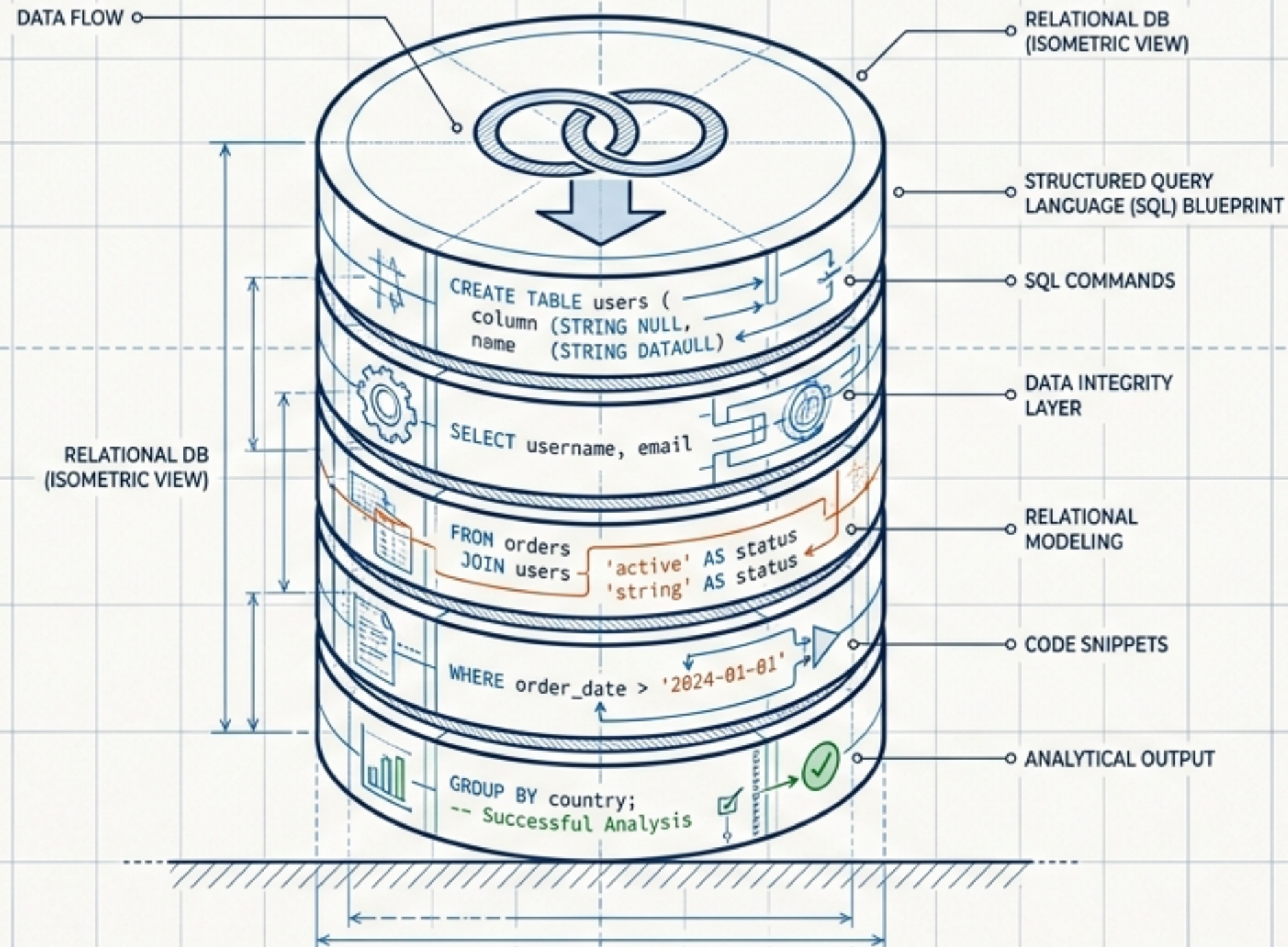


MESTRES DO SQL

Do básico estrutural às análises complexas de relacionamentos.
Um guia visual e prático.

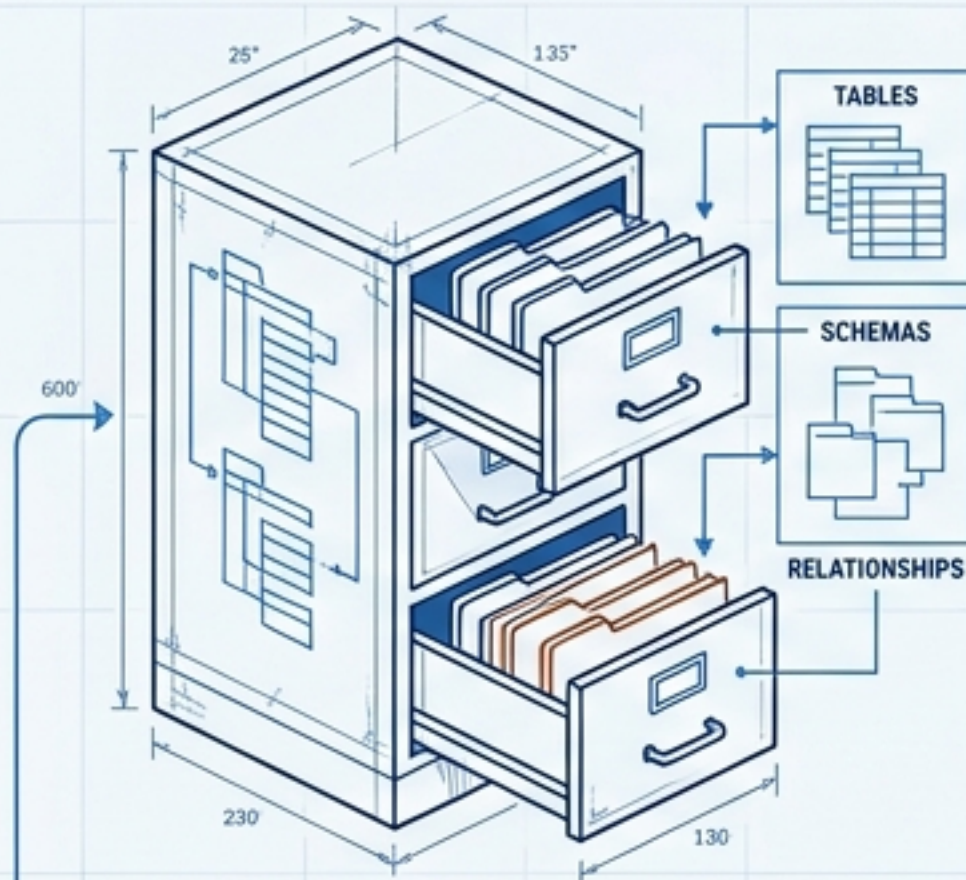


O Ecossistema de um Banco de Dados Relacional

O SQL opera em três frentes essenciais. Primeiro, definimos a arquitetura. Depois, populamos com informações. Por fim, extraímos inteligência.

O Armário e as Pastas (Estrutura)

DDL (Data Definition Language)

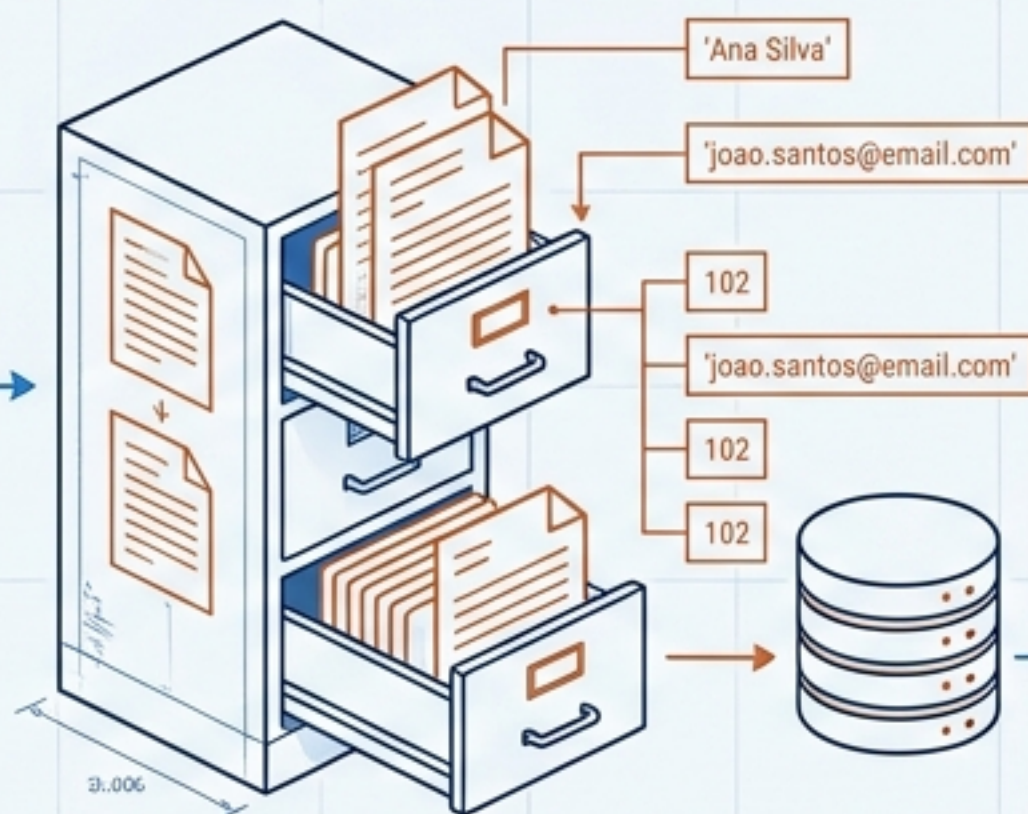


CODE SNIPPET (DDL)

```
CREATE TABLE clientes (  
  id INT PRIMARY KEY,  
  nome VARCHAR(100),  
  email VARCHAR(255) UNIQUE  
);  
  
-- Comentário: Define a estrutura da tabela clientes.
```

As Fichas de Papel (Dados)

DML (Data Manipulation Language)

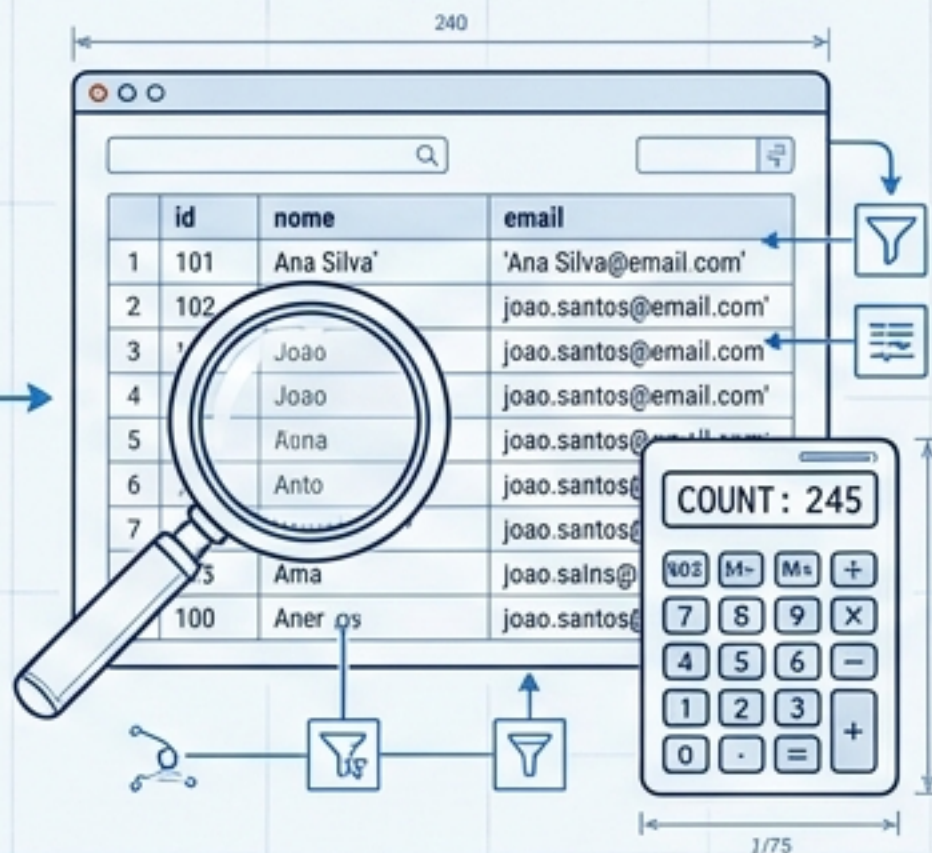


CODE SNIPPET (DML)

```
INSERT INTO clientes (id, nome, email)  
VALUES (101, 'Ana Silva', 'ana.silva@email.com');  
  
UPDATE clientes SET email = 'ana.novo@email.com'  
WHERE id = 101;  
  
-- Comentário: Insere e atualiza os dados nas tabelas.
```

A Lupa e a Calculadora (Consultas)

DQL (Data Query Language)

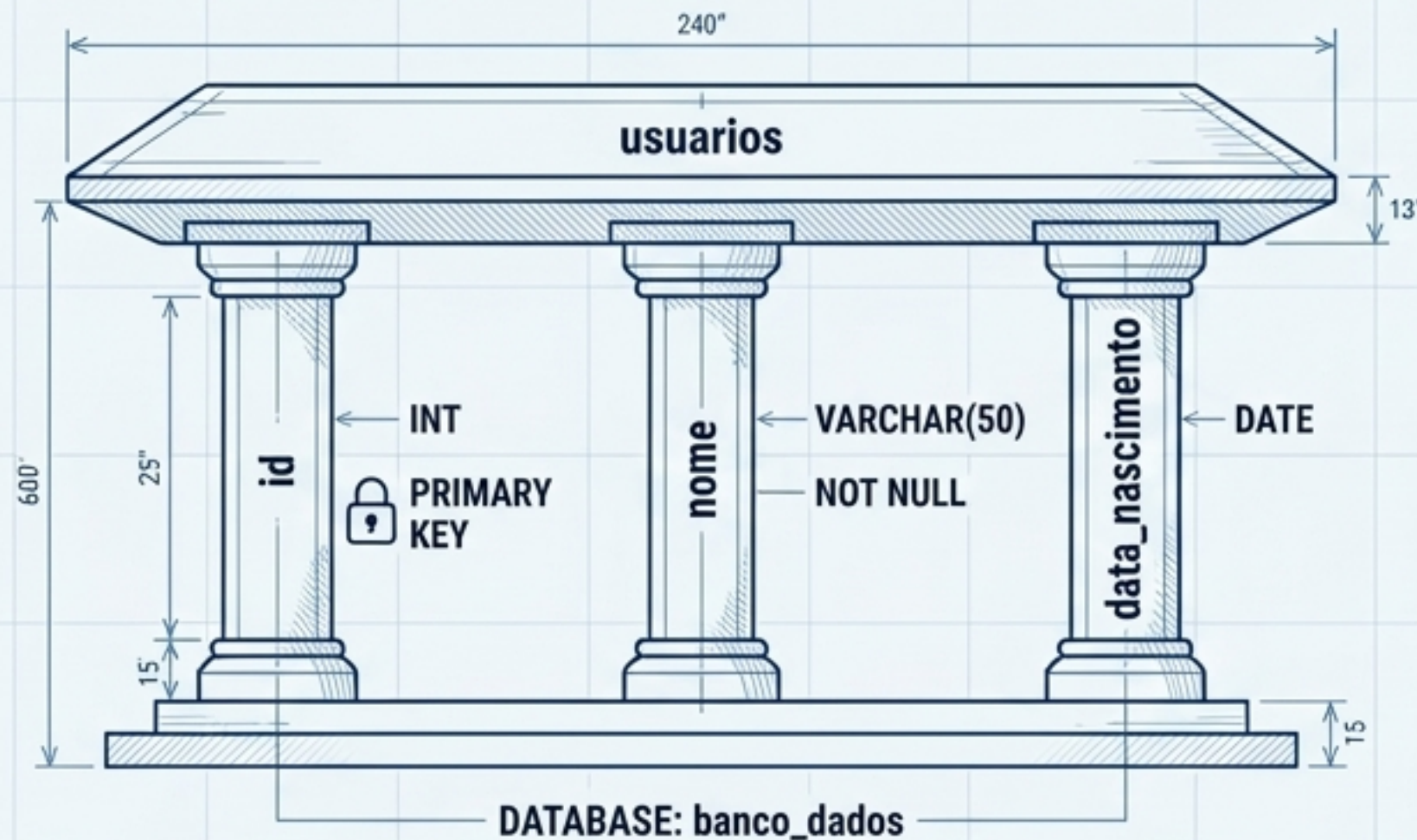


CODE SNIPPET (DQL)

```
SELECT nome, email  
FROM clientes  
WHERE id > 100  
ORDER BY nome ASC;  
  
-- Comentário: Consulta e extrai dados com base em  
-- critérios.
```

Construindo os Alicerces com DDL e Tipos de Dados

Antes de inserir dados, precisamos definir a fundação. Utilizamos **CREATE DATABASE** para o repositório e **CREATE TABLE** para as entidades.



```
1 CREATE DATABASE banco_dados;  
2  
3 CREATE TABLE usuarios (  
4     id INT PRIMARY KEY,  
5     nome VARCHAR(50) NOT NULL,  
6     data_nascimento DATE  
7 );
```

Tipos de Dados e Restrições



INT / SMALLINT

Números inteiros
(ex: idade).



VARCHAR(50)

Textos variáveis e
eficientes (ex: e-mail).



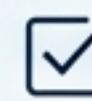
DATE

Datas estruturadas
(ex: nascimento).



PRIMARY KEY

Identificador único.



NOT NULL

Campo de preenchimento
obrigatório.

Desafios Práticos: Nível Básico

1



Desafio 1: Criação de Banco de Dados

Crie um banco de dados chamado `pizzaria`.

ooo

```
CREATE DATABASE pizzaria;
```



-- Comentário: Inicia a estrutura.

2



Desafio 2: Criação de Tabela

Crie uma tabela `clientes` contendo `id` (chave primária), `nome` (texto até 50 caracteres, obrigatório) e `email` (texto até 50 caracteres, único).

ooo

```
CREATE TABLE clientes (  
  id INT PRIMARY KEY,  
  nome VARCHAR(50) NOT NULL,  
  email VARCHAR(50) UNIQUE  
);
```



-- Comentário: Define as colunas e restrições.

3



Desafio 3: Alteração de Estrutura

O cliente mudou de ideia. Use o comando `ALTER TABLE` para adicionar uma coluna `idade` do tipo numérico pequeno (`SMALLINT`).

ooo

```
ALTER TABLE clientes ADD idade  
SMALLINT;
```



-- Comentário: Adiciona uma nova coluna.

Resoluções: Estruturas e Tabelas



```
1 | CREATE DATABASE pizzaria;
```



```
1 | CREATE TABLE clientes (  
2 |     id SERIAL PRIMARY KEY,  
3 |     nome VARCHAR(50) NOT NULL,  
4 |     email VARCHAR(50) UNIQUE  
5 | );
```

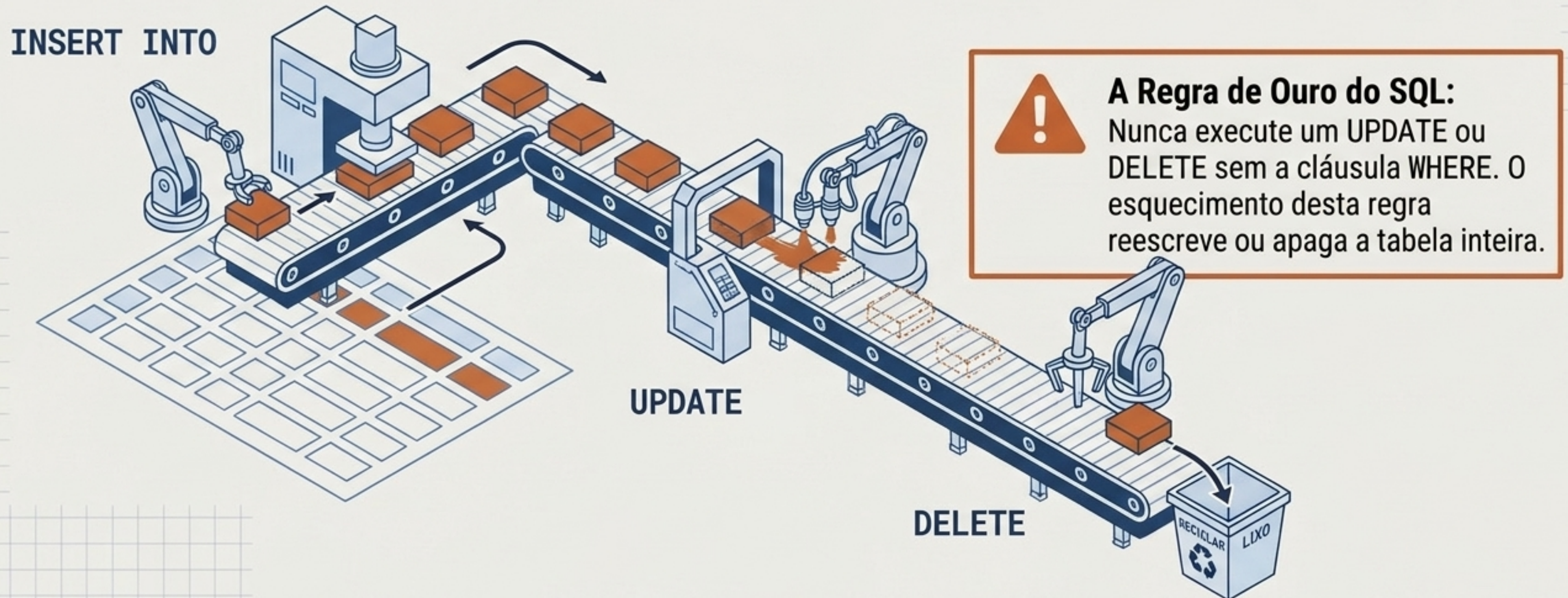
Nota: `SERIAL` (ou `AUTO\_INCREMENT`) gera IDs automaticamente. `UNIQUE` impede e-mails duplicados em nível estrutural.



```
1 | ALTER TABLE clientes ADD COLUMN idade SMALLINT;
```

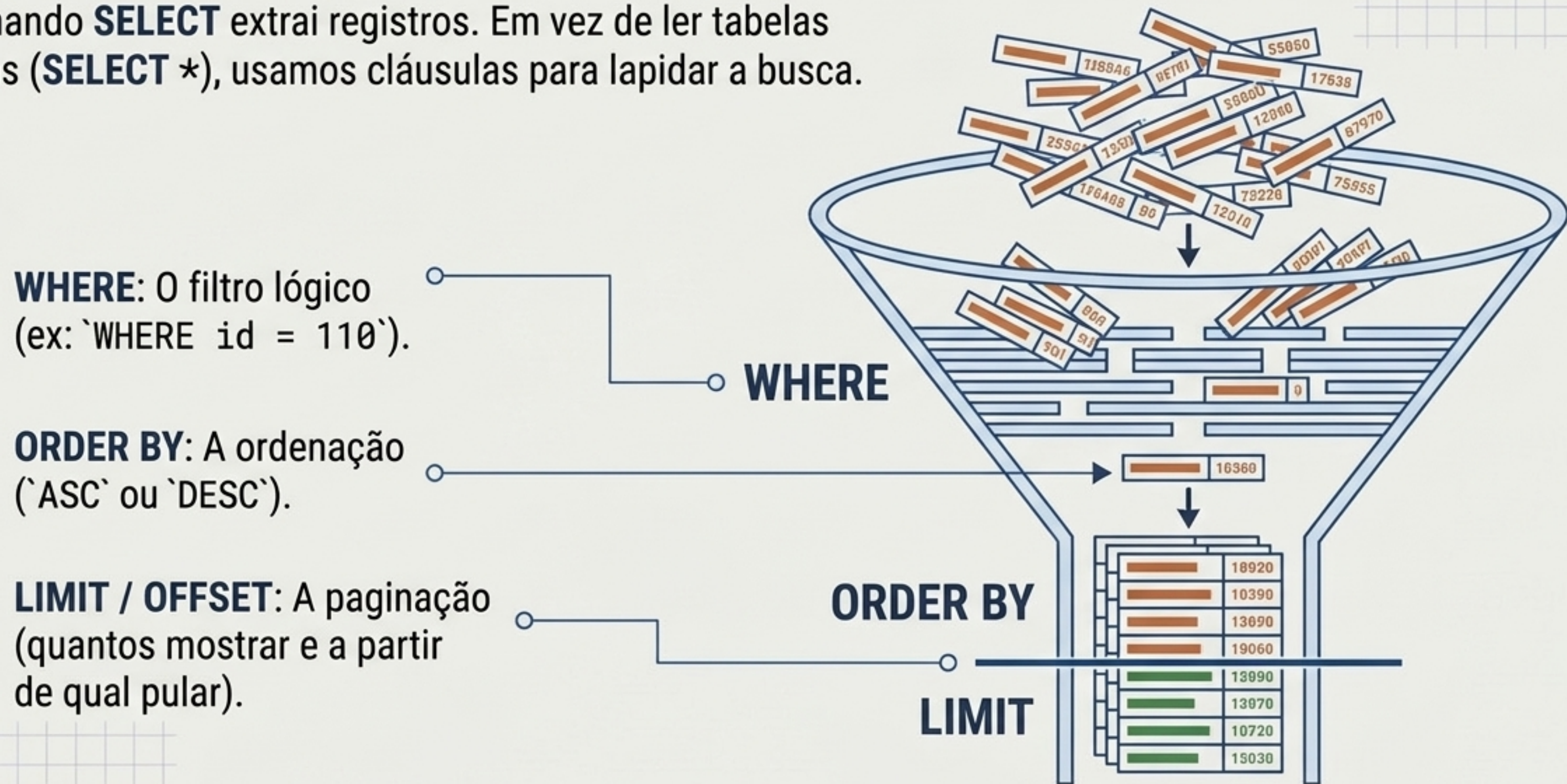
Dando Vida aos Dados (e evitando desastres)

A Manipulação de Dados (DML) injeta vida na sua estrutura. O **INSERT INTO** adiciona novas linhas. O **UPDATE** altera dados e o **DELETE** os remove.



A Arte de Perguntar: O Funil de Consulta

O comando **SELECT** extrai registros. Em vez de ler tabelas inteiras (**SELECT ***), usamos cláusulas para lapidar a busca.



Matriz de Filtros Avançados

O Rastreador (LIKE)	O Intervalo (BETWEEN)	O Catálogo (IN)
		
<pre>WHERE nome LIKE 'M%'</pre>	<pre>WHERE preco BETWEEN 1000 AND 1500</pre>	<pre>WHERE cor IN ('Azul', 'Vermelho')</pre>
Busca padrões de texto. `%` substitui qualquer sequência, `_` substitui um único caractere.	Captura faixas contínuas, ideal para datas e valores financeiros. Inclui os limites.	Substitui múltiplos `OR`. Filtra registros que correspondam a qualquer valor dentro de uma lista exata.

Desafios Práticos: Nível Intermediário

1

Insira dois novos registros na tabela `clientes` com uma única instrução SQL.

2

Selecione o `nome` e o `email` de todos os clientes cujo nome comece com a letra "A".

3

Atualize o e-mail do cliente com ID igual a 4 para `"novo@email.com"`. (Lembre-se da regra de ouro!)

Resoluções: Manipulação e Filtros



```
INSERT INTO clientes (nome, email)
VALUES ('Ana', 'ana@email.com'),
      ('Beto', 'beto@email.com');
```

Dica: O comando `VALUES` aceita múltiplos grupos de dados separados por vírgula para otimizar e economizar processamento.



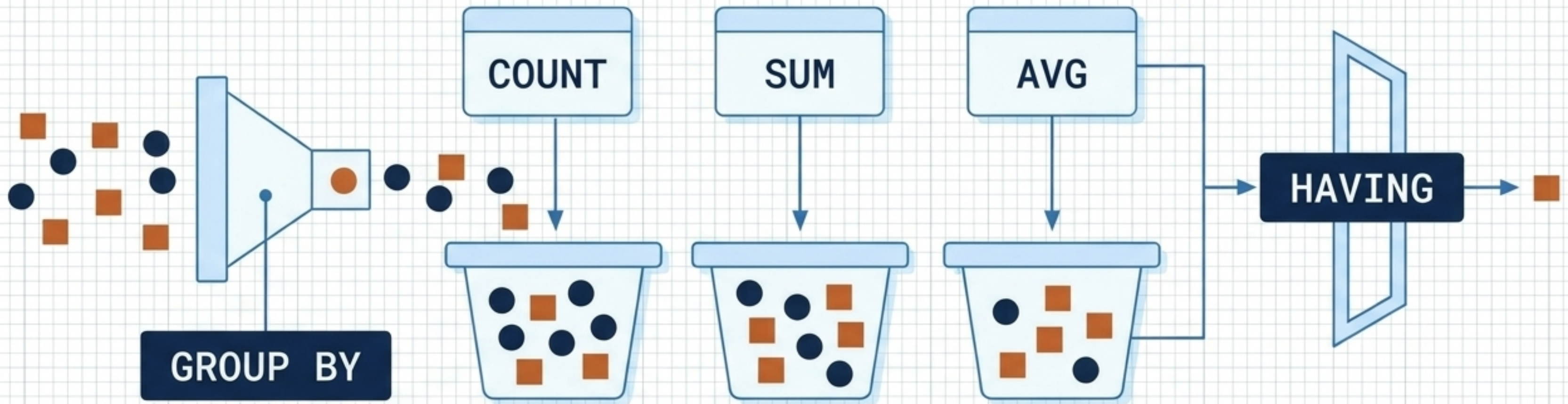
```
SELECT nome, email FROM clientes
WHERE nome LIKE 'A%';
```



```
UPDATE clientes SET email = 'novo@email.com'
WHERE id = 4;
```

Síntese de Dados: Agrupamento e Matemática

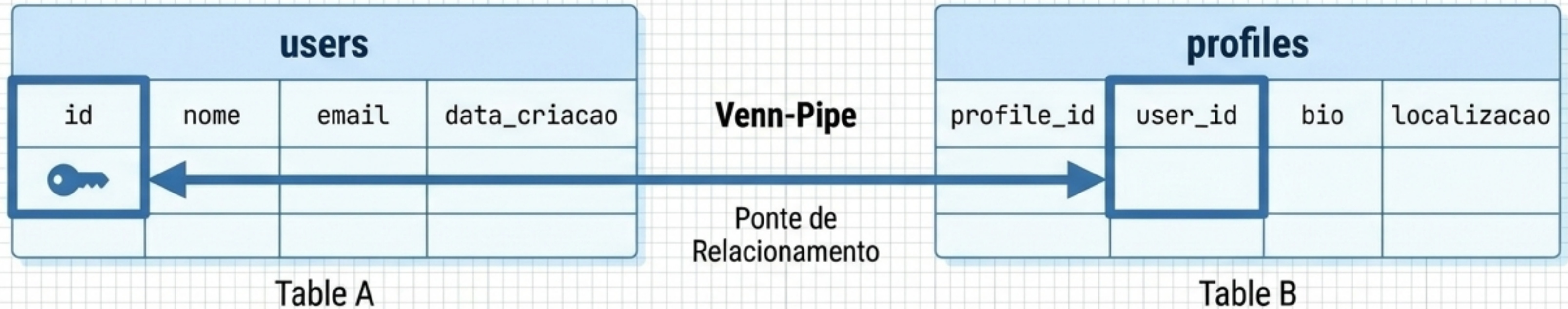
Funções de agregação transformam milhares de linhas em insights únicos.



Distinção Crucial: O `WHERE` filtra os dados individuais ANTES de entrarem nos baldes. O `HAVING` filtra os grupos inteiros DEPOIS da matemática ser calculada.

Conectando os Pontos: O Poder do INNER JOIN

Bancos relacionais evitam a duplicação espalhando dados. O JOIN é a ponte que une tabelas.



```
SELECT u.nome, p.bio  
FROM users u  
INNER JOIN profiles p  
ON u.id = p.user_id;
```

Aliases (`u` e `p`) salvam tempo de digitação e impedem ambiguidades quando tabelas diferentes possuem colunas com o mesmo nome.

Desafios Práticos: Nível Avançado

1

Descubra a média (AVG) de idade de todos os clientes registrados.

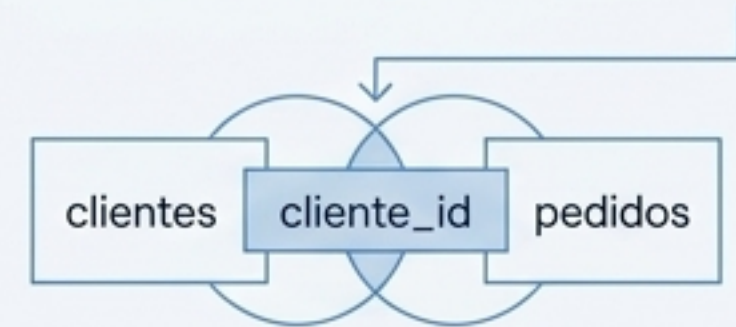
2

Conte quantos cursos existem agrupados por ano, mas mostre apenas os anos que tiveram mais de 3 cursos lançados.



3

Escreva um INNER JOIN para mostrar o nome do cliente e o número do seu pedido, conectando a tabela clientes à tabela pedidos via cliente_id.



Resoluções: Análise e Relacionamentos

Code Block 1:



```
1 SELECT AVG(idade) AS media_idade
2 FROM clientes;
```

Diagram: A blue arrow points from the text 'A palavra-chave `AS` cria um `alias` limpo e legível para a coluna de saída agregada no resultado final.' to the 'AS' keyword in the SQL query. Another blue arrow points from the 'media_idade' alias to the same text box.

A palavra-chave **`AS`** cria um 'alias' limpo e legível para a coluna de saída agregada no resultado final.

Code Block 2:



```
1 SELECT ano, COUNT(*) as total_cursos
2 FROM cursos
3 GROUP BY ano
4 HAVING COUNT(*) > 3;
```

Code Block 3:



```
1 SELECT c.nome, p.numero_pedido
2 FROM clientes c
3 INNER JOIN pedidos p
4 ON c.id = p.cliente_id;
```

O Cheat Sheet Definitivo

Arquitetura (DDL)

CREATE DATABASE

CREATE TABLE

ALTER

DROP

Tipos: INT
 VARCHAR(50)
 DATE

Restrições: PRIMARY KEY
 NOT NULL
 UNIQUE

Ação (DML)

INSERT INTO tabela (col1)
VALUES (val1);

UPDATE tabela
SET col1 = val1
WHERE id = 1;

DELETE FROM tabela
WHERE id = 1;

Inteligência (DQL)

SELECT col1, col2
FROM tabela
WHERE
 (LIKE, BETWEEN, IN)
ORDER BY col1 ASC|DESC
LIMIT 10
GROUP BY col1
HAVING COUNT(*) > 1
INNER JOIN tabela2
ON tab1.id = tab2.id

Guarde este mapa. Ele é a chave para dominar seus dados.